P2587R0
`to_string` or not `to_string`

Published Proposal, 2022-05-14

Author:
Victor Zverovich

Audience:
LEWG

Project:
ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Table of Contents

"Though this be madness, yet there is method in ’t." — Polonius

§ 1. Introduction

C++11 introduced a set of `std::to_string` overloads for integral and floating-point types. Fortunately for integral and unfortunately for floating-point overloads they are all defined in terms of `sprintf` inconsistently with C++ formatted output functions ([N4910]). Additionally, the choice of the floating-point format makes `std::to_string` of very limited use in practice. This paper proposes fixing these issues while retaining existing semantics of integral overloads.

§ 2. Examples

Consider the following example:

```
auto loc = std::locale("uk_UA.UTF-8");
std::locale::global(loc);
std::cout.imbue(loc);
setlocale(LC_ALL, "C");

std::cout << "iostreams:\n";
std::cout << 1234 << "\n";
std::cout << 1234.5 << "\n";

std::cout << "\nto_string:\n";
std::cout << std::to_string(1234) << "\n";
std::cout << std::to_string(1234.5) << "\n";

setlocale(LC_ALL, "uk_UA.UTF-8");

std::cout << "\nto_string (uk_UA.UTF-8 C locale):\n";
std::cout << std::to_string(1234) << "\n";
std::cout << std::to_string(1234.5) << "\n";
```

It prints:

```
iostreams:
1 234
1 234,5

to_string:
1234
1234.500000
```

```
to_string (uk_UA.UTF-8 C locale):
1234
1234,500000
```

Since `std::to_string` uses the global C locale and no grouping the integral overloads are effectively unlocalized. The output of floating-point overloads is inconsistent with that of iostreams because the former takes the decimal point from the global C locale and doesn't do grouping.

Additionally, due to an unfortunate choice of the fixed format in the floating-point overloads they are only useful for numbers in a limited exponent range. For example:

```
std::cout << std::to_string(std::numeric_limits<double>::max());
```

prints

```
179769313486231570814527423731704356798070567525844996598917476803157260780
387605895586327668781715404589535143824642343213268894641827684675467035375
604991057655128207624549009038932894407586850845513394230458323690322294816
559332123348274797826204144723168738177180919299881250404026184124858368.00
```

(line breaks inserted for readability)

Here only the first 17 digits are meaningful, the next 292 are so-called "garbage" digits ([DRAGON]). And finally we have 6 meaningless zeros after a possibly localized decimal point.

Formatting of small floating-point numbers is even less useful. For example:

```
std::cout << std::to_string(-1e-7);
```

prints

```
-0.000000
```

§ 3. Proposal

Redefine `std::to_string` in terms of `std::format` which in turn uses `std::to_chars` making more explicit the fact that integral overloads are unlocalized and changing the format of floating-point overloads to also be unlocalized and use the shortest decimal representation.

The following table shows the changes in output for the following code:

```
setlocale(LC_ALL, "C");
auto output = std::to_string(input);
```

input	output	
	before	**after**
42	42	42
0.42	0.420000	0.42
-1e-7	-0.000000	-1e-7
1.7976931348623157e+308	179769313486231570814527423731704356798070567525844996598917476803157260780028538760589558632766878171540458953514382464234321326889464182768467546703537516986049910576551282076245490090389328944075868508455133942304583236903222948165808559332123348274797826204144723168738177180919299881250404026184124858368.000000	1.7976931348623157e+308

and similarly with the global C locale set:

```
setlocale(LC_ALL, "uk_UA.UTF-8");
auto output = std::to_string(input);
```

input	output	
	before	**after**
12345	12345	12345
1234.5	1234,500000	1234.5

§ 4. Impact on existing code

This change will affect the output of `std::to_string` with floating-point arguments. In most cases it will result in a more precise and/or shorter output. In cases where the C locale is explicitly set the decimal point will no longer be localized.

§ 5. Implementation

{fmt} implements proposed changes in `fmt::to_string`.

§ 6. Wording

Modify subsection "Numeric conversions [string.conversions]":

```
string to_string(int val);
string to_string(unsigned val);
string to_string(long val);
string to_string(unsigned long val);
string to_string(long long val);
string to_string(unsigned long long val);
string to_string(float val);
string to_string(double val);
string to_string(long double val);
```

~~*7 Returns:* Each function returns a string object holding the character representation of the value of its argument that would be generated by calling `sprintf(buf, fmt, val)` with a format specifier of `"%d"`, `"%u"`, `"%ld"`, `"%lu"`, `"%lld"`, `"%llu"`, `"%f"`, `"%f"`, or `"%Lf"`, respectively, where `buf` designates an internal character buffer of sufficient size.~~

7 Returns: `format("{}", val)`.

§ References

§ Informative References

[DRAGON]

Guy L. Steele Jr.; Jon L White. How to Print Floating-Point Numbers Accurately. URL: https://fmt.dev/papers/p372-steele.pdf

[N4910]

Thomas Köppe. Working Draft, Standard for Programming Language C++. URL: https://wg21.link/n4910